

ProPresenter Automation System — Complete Project Requirements

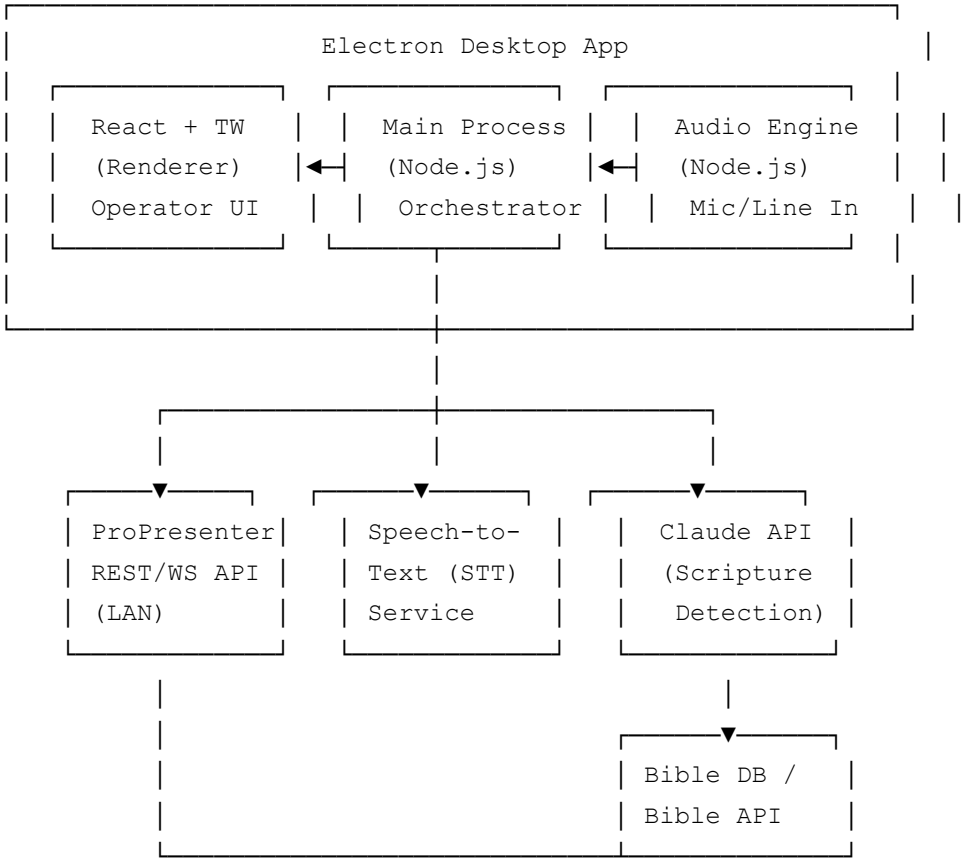
1. Project Overview

Project Name: ProPresenter Church Automation System (working title: "ProAutomate")

Vision: A desktop application that connects to ProPresenter via its API, listens to live audio during church services, intelligently detects scripture references and quotes from the minister’s speech, and automatically projects the corresponding Bible verses — while also providing tools for searching, importing, and managing song lyrics.

Target Users: Church media/tech teams who currently operate ProPresenter manually.

2. System Architecture



3. Requirements

3.1 Hardware Requirements

Item	Requirement	Notes
Computer	Windows 10/11 or macOS 12+	Same machine as ProPresenter or on same LAN
RAM	8 GB minimum, 16 GB recommended	STT processing + Electron overhead
Audio Input	USB audio interface or direct soundboard feed	Line-level input from mixing console is strongly preferred over a room mic for accuracy
Network	Same LAN as ProPresenter machine	ProPresenter API is local REST/WebSocket
Internet	Stable broadband connection	Required for STT API, Claude API, and lyrics search

3.2 Software Requirements

Software	Purpose	Notes
ProPresenter 7.9+	Presentation software	Must have Network API enabled in settings
Node.js 20 LTS+	Runtime	Electron's backend
CCLI License	Song lyrics legal compliance	Your church likely has one already

3.3 API Keys / Accounts Needed

Service	Purpose	Estimated Cost
Deepgram API	Real-time speech-to-text	~\$0.0077/min streaming (pay-as-you-go). ~\$4.60/hr
Anthropic Claude API	Scripture detection & NLP intelligence	~\$3/M input tokens, \$15/M output tokens (Sonnet)

Bible API (api.bible or bible-api.com)	Scripture text lookup	Free for non-commercial use
SongSelect / CCLI account	Worship song lyrics	Included with church CCLI subscription

4. Feature Specification

4.1 Core Features (MVP)

F1: ProPresenter Connection Manager

- Auto-discover ProPresenter instances on the local network
- Connect via REST API (HTTP, port 50000 by default) and WebSocket for real-time events
- Display connection status (connected / disconnected / reconnecting)
- Show current active presentation, slide index, and playlist
- Support ProPresenter 7.9+ official REST API (231 endpoints across 27 API groups)

F2: Real-Time Audio Transcription

- Capture live audio from a selectable input device (mic, line-in, virtual audio device)
- Stream audio to Deepgram's real-time STT API via WebSocket
- Display running transcription with interim (partial) and final results
- Visual audio level meter to confirm signal is being received
- Configurable audio source selection (enumerate all system audio devices)
- Support for direct soundboard feed (preferred) and room microphone (fallback)

F3: Automatic Scripture Detection

- Continuously analyze transcription output using Claude API
- Detect three types of scripture references:
 - **Explicit references:** "Turn your Bibles to Romans 8:28" → Romans 8:28

- **Direct quotes:** "For God so loved the world..." → John 3:16
- **Paraphrased references:** "Paul tells us to rejoice always" → 1 Thessalonians 5:16
- Maintain a sliding context window (last 60–120 seconds of transcript) for better detection accuracy
- Return structured data: book, chapter, verse(s), confidence score, translation preference

F4: Bible Verse Lookup & Display

- Query a Bible API or local database for the detected verse(s)
- Support multiple translations: KJV, NIV, ESV, NKJV, NLT, AMP, MSG (configurable default)
- Format verse text for projection (large text, verse reference, translation label)
- Push formatted scripture to ProPresenter as a new presentation or a pre-built template slide
- Support verse ranges (e.g., Romans 8:28-30)

F5: Operator Control Panel

- **Suggestion queue:** Detected scriptures appear as suggestions with confidence indicators
- **One-click approve:** Operator taps to project the suggested verse immediately
- **One-click dismiss:** Operator dismisses false positives
- **Auto mode toggle:** When confidence is high (configurable threshold), project automatically without operator approval
- **Manual search:** Operator can type any scripture reference and project it instantly
- **Emergency clear:** One button to clear all projected content immediately

F6: Song Lyrics Search & Import

- Search for worship songs by title, artist, or lyrics snippet
- Import song lyrics directly into ProPresenter's library/playlist
- Format lyrics into projection-ready slides (verse, chorus, bridge sections)

- Support importing from SongSelect (.usr files) if the church has a CCLI account
- Manual lyrics entry with auto-formatting into slide groups

4.2 Enhanced Features (Post-MVP)

F7: Service Planning Integration

- Pre-load a service order / run sheet for the day
- Know which songs are planned so scripture detection can prioritize sermon segments
- Track service progress (worship → announcements → sermon → altar call)

F8: Sermon Notes / Transcript Export

- Export the full transcription of the sermon as a text file, markdown, or Word document
- Auto-tag detected scriptures within the transcript
- Timestamp each section for easy review

F9: Historical Analytics Dashboard

- Track which scriptures are referenced most frequently
- Track which songs are used most often
- Provide reports for the worship/tech team lead

F10: Multi-Translation Comparison

- When a verse is detected, show it side-by-side in 2-3 translations
- Let the operator choose which translation to project

F11: Custom Slide Templates

- Let users create and save ProPresenter slide templates for scriptures
- Configure fonts, backgrounds, layouts, and animations
- Map templates to different service segments (e.g., sermon template vs. worship template)

F12: Keyword Highlighting & Alerts

- Allow configuring custom keywords or phrases to watch for (e.g., "altar call", "let us pray")
- Trigger pre-configured ProPresenter actions (macros, looks, props) when keywords are detected

F13: Stage Display Integration

- Show upcoming/suggested scriptures on the ProPresenter stage display for the minister
- Display sermon timer and transcript on stage screens

F14: Team Collaboration

- Multiple operators can connect to the same session
- One person monitors scripture detection while another handles song lyrics
- Role-based views (producer view, operator view, read-only view)

5. Tech Stack

5.1 Application Shell

Layer	Technology	Rationale
Desktop Framework	Electron (with electron-forge or electron-vite)	Bundles Node.js + Chromium. Your React/Node skills transfer directly.
Frontend	React 18+ with TypeScript	You already know it. Component-based UI is perfect for the operator panel.
Styling	Tailwind CSS	You already know it. Rapid UI prototyping.
State Management	Zustand or Redux Toolkit	Zustand is simpler and works beautifully with Electron.
Build Tool	Vite	Fast HMR, great DX with electron-vite.

5.2 Backend / Main Process

Component	Technology	Rationale

IPC Layer	Electron IPC (ipcMain / ipcRenderer)	Communication between main process and renderer.
ProPresenter Client	Custom HTTP/WebSocket client (axios + ws)	The Pro7.9+ API is standard REST + chunked streaming.
Audio Capture	node-record-lpcm16 or Web Audio API (renderer)	Capture raw audio from system input devices.
STT Client	Deepgram Node SDK (@deepgram/sdk)	Native WebSocket streaming, Node.js SDK available, sub-300ms latency.
Scripture Detection	Anthropic Claude SDK (@anthropic-ai/sdk)	Best-in-class NLP for understanding spoken biblical references.
Bible Database	SQLite (via better-sqlite3) + API fallback	Local SQLite for instant lookups; API.Bible or bible-api.com as fallback for additional translations.
Song Lyrics	Web scraping / SongSelect import	Parse .usr files from SongSelect; manual entry as fallback.
Settings/Config	electron-store	Persistent JSON config for API keys, preferences, audio device selection.
Logging	electron-log	Unified logging across main and renderer processes.

5.3 External Services

Service	SDK / Integration	Purpose
Deepgram	@deepgram/sdk	Real-time speech-to-text via WebSocket
Claude (Anthropic)	@anthropic-ai/sdk	Scripture reference detection and NLP
API.Bible	REST (fetch)	Bible verse text retrieval (2500+ versions, 1600+ languages)
bible-api.com	REST (fetch)	Lightweight fallback Bible API (no auth required)
	Custom	Slide control, library management, playlist

ProPresenter	REST/WebSocket client	manipulation
--------------	-----------------------	--------------

5.4 Development & Quality

Tool	Purpose
TypeScript	Type safety across the entire codebase
ESLint + Prettier	Code quality and formatting
Vitest	Unit testing
Playwright	E2E testing for the Electron app
GitHub Actions	CI/CD pipeline
electron-builder	Packaging for Windows (.exe) and macOS (.dmg)

6. Data Models

6.1 Scripture Reference

```
interface ScriptureReference {
  book: string;          // "Romans"
  chapter: number;       // 8
  verseStart: number;    // 28
  verseEnd?: number;     // 30 (for ranges)
  translation: string;   // "KJV"
  confidence: number;    // 0.0 - 1.0
  detectionType: 'explicit' | 'quote' | 'paraphrase';
  sourceText: string;    // The transcript snippet that triggered detection
  timestamp: number;     // When it was detected
}
```

6.2 Transcription Segment

```
interface TranscriptionSegment {
  text: string;
  isFinal: boolean;
  confidence: number;
  timestamp: number;
```

```
    duration: number;
}
```

6.3 ProPresenter Connection

```
interface ProPresenterConfig {
  host: string;          // "192.168.1.100"
  port: number;          // 50000
  password?: string;
  version: string;       // "7.9+"
  connected: boolean;
}
```

6.4 Song / Lyrics

```
interface Song {
  id: string;
  title: string;
  artist: string;
  ccliNumber?: string;
  sections: SongSection[];
  copyright?: string;
}

interface SongSection {
  type: 'verse' | 'chorus' | 'bridge' | 'pre-chorus' | 'tag' | 'intro' | 'outro';
  label: string;      // "Verse 1", "Chorus"
  lines: string[];
}
```

7. ProPresenter API Integration Details

ProPresenter 7.9+ exposes an official REST API (documented at openapi.propresenter.com) with 231 endpoints across 27 groups. Key endpoints for this project:

7.1 Endpoints You Will Use

Endpoint Category	Key Actions	Purpose
/v1/presentation	GET active, trigger slide	Control what's currently displayed

<code>/v1/playlist</code>	GET all, GET items, trigger item	Navigate playlists and trigger content
<code>/v1/library</code>	GET all items	Browse the ProPresenter library
<code>/v1/status</code>	GET connection status, subscribe to updates	Monitor ProPresenter state
<code>/v1/stage/message</code>	PUT message	Display text on stage screens
<code>/v1/clear</code>	POST clear layer	Clear projected content
<code>/v1/message</code>	GET/POST messages	Control message overlays
<code>/v1/macro</code>	GET/POST macros	Trigger pre-configured macros
<code>/v1/timer</code>	GET/POST timers	Control countdown/elapsed timers
<code>/v1/look</code>	GET/POST looks	Change audience screen looks

7.2 Connection Method

REST API: `http://{pro_presenter_ip}:{port}/v1/{endpoint}`

Streaming: GET with chunked transfer encoding for `/v1/status/updates`

TCP/IP: JSON-over-TCP as an alternative (one JSON object per line, CRLF termin

7.3 Key Considerations

- The API uses standard HTTP GET/POST/PUT/DELETE
 - Streaming updates use chunked transfer encoding (not WebSocket for the new API)
 - The older WebSocket-based remote/stagedisplay protocol (pre-7.9) is also available but undocumented
 - Be careful with invalid messages — they can crash ProPresenter
 - The API documentation is viewable in-app via the Network settings tab
-

8. Scripture Detection Pipeline (Detailed)

This is the most technically complex part of the system. Here is the detailed pipeline:

Step 1: Audio Capture

```
Soundboard Line Out → Audio Interface → Node.js Audio Capture
                                   |
                                   PCM audio stream (16-bit, 16kHz mono)
```

Step 2: Speech-to-Text

```
PCM stream → Deepgram WebSocket → Interim transcripts (fast, partial)
                                   → Final transcripts (accurate, complete)
```

Step 3: Context Buffering

```
Final transcripts → Rolling buffer (last 90 seconds)
                  → Sentence boundary detection
                  → Trigger scripture analysis every ~5-10 seconds
                     or on explicit trigger phrases
```

Step 4: Scripture Detection (Claude API)

Send the buffered transcript to Claude with a carefully crafted prompt:

```
System: You are a scripture detection assistant for a live church service.
Analyze the following transcript and identify any Bible scripture references,
whether explicit ("Turn to John 3:16"), direct quotes ("For God so loved the world")
or paraphrased references ("Paul tells us to be content in all situations").
```

```
Return ONLY a JSON array. If no scripture is found, return [].
```

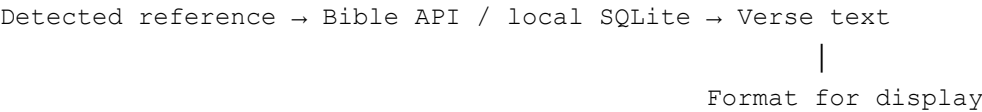
```
Each entry should have:
```

- book, chapter, verseStart, verseEnd (if range)
- confidence (0.0 to 1.0)
- detectionType: "explicit" | "quote" | "paraphrase"
- sourceText: the transcript snippet

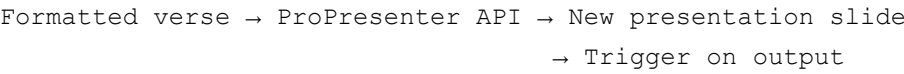
```
Transcript:
```

```
"{buffered_transcript}"
```

Step 5: Bible Lookup



Step 6: ProPresenter Push



Latency Budget

Stage	Target Latency	Notes
Audio capture	<50ms	Near instant
STT (Deepgram)	<300ms	Deepgram’s streaming target
Context buffer	3-8 seconds	Accumulate enough speech context
Claude API call	1-2 seconds	Depends on prompt/model size
Bible lookup	<200ms	Local SQLite is instant; API adds latency
ProPresenter push	<200ms	LAN REST call
Total end-to-end	~5-10 seconds	From minister speaking to verse on screen

This is acceptable because ministers typically reference a verse and then pause or repeat it, giving the system a natural window.

9. Getting Started Guide

9.1 Prerequisites

```
# Install Node.js 20 LTS
# Install Git

# Verify
node --version # v20.x.x
```

```
npm --version      # 10.x.x
```

9.2 Project Scaffolding

```
# Option 1: electron-vite (recommended)
npm create @quick-start/electron proautomate -- --template react-ts
```

```
# Option 2: electron-forge with Vite + React
npx create-electron-app proautomate --template=vite-typescript
```

9.3 Install Core Dependencies

```
cd proautomate

# Electron + React essentials (most already scaffolded)
npm install react react-dom
npm install -D @types/react @types/react-dom tailwindcss postcss autoprefixer

# ProPresenter API communication
npm install axios ws
npm install -D @types/ws

# Speech-to-text
npm install @deepgram/sdk

# Scripture detection
npm install @anthropic-ai/sdk

# Bible database
npm install better-sqlite3
npm install -D @types/better-sqlite3

# Audio capture
npm install node-record-lpcm16
# OR use Web Audio API in the renderer process

# App utilities
npm install electron-store electron-log zustand uuid
npm install -D @types/uuid
```

9.4 Project Structure

```
proautomate/
```

```

└─ electron/                                # Electron main process
|   └─ main.ts                             # App entry point, window creation
|   └─ preload.ts                          # Context bridge for IPC
|   └─ ipc/                                # IPC handler modules
|       └─ propresenter.ipc.ts
|       └─ audio.ipc.ts
|       └─ scripture.ipc.ts
|       └─ lyrics.ipc.ts
|   └─ services/                          # Backend services (main process)
|       └─ propresenter/
|           └─ ProPresenterClient.ts
|           └─ ProPresenterDiscovery.ts
|           └─ SlideBuilder.ts
|       └─ audio/
|           └─ AudioCapture.ts
|           └─ AudioManager.ts
|       └─ stt/
|           └─ DeepgramClient.ts
|           └─ TranscriptionBuffer.ts
|       └─ scripture/
|           └─ ScriptureDetector.ts      (Claude API)
|           └─ BibleLookup.ts
|           └─ BibleDatabase.ts          (SQLite)
|       └─ lyrics/
|           └─ LyricsSearchService.ts
|           └─ SongSelectImporter.ts
|   └─ db/
|       └─ bible.sqlite                    # Local Bible database
└─ src/                                    # React renderer process
    └─ App.tsx
    └─ main.tsx
    └─ components/
        └─ layout/
            └─ Sidebar.tsx
            └─ Header.tsx
            └─ StatusBar.tsx
        └─ dashboard/
            └─ Dashboard.tsx
            └─ ConnectionStatus.tsx
            └─ AudioMeter.tsx
        └─ scripture/
            └─ ScriptureSuggestions.tsx
            └─ ScriptureCard.tsx
            └─ ManualSearch.tsx
            └─ TranslationPicker.tsx
        └─ transcription/
            └─ LiveTranscript.tsx

```

```

| | | └─ TranscriptHistory.tsx
| | └─ lyrics/
| |   └─ LyricsSearch.tsx
| |   └─ LyricsEditor.tsx
| |   └─ PlaylistManager.tsx
| | └─ operator/
| |   └─ OperatorPanel.tsx
| |   └─ QuickActions.tsx
| |   └─ EmergencyClear.tsx
| | └─ settings/
| |   └─ SettingsPage.tsx
| |   └─ AudioSettings.tsx
| |   └─ ProPresenterSettings.tsx
| |   └─ APIKeysSettings.tsx
| |   └─ ScriptureSettings.tsx
| └─ hooks/
|   └─ useProPresenter.ts
|   └─ useTranscription.ts
|   └─ useScripture.ts
|   └─ useAudio.ts
| └─ stores/
|   └─ appStore.ts
|   └─ scriptureStore.ts
|   └─ transcriptionStore.ts
| └─ lib/
|   └─ ipc.ts # Type-safe IPC calls from renderer
|   └─ types.ts # Shared TypeScript interfaces
└─ resources/ # App icons, assets
└─ package.json
└─ electron.vite.config.ts
└─ tailwind.config.ts
└─ tsconfig.json
└─ README.md

```

9.5 Build Order (What to Build First)

Phase 1 — Foundation (Week 1-2)

1. Scaffold Electron + React + Tailwind app
2. Set up Electron IPC bridge (preload script, context bridge)
3. Build the settings page (API keys, ProPresenter IP/port, audio device selection)
4. Build the ProPresenter connection manager (connect, disconnect, show status)
5. Test basic ProPresenter API calls (get library, get playlists, trigger a slide)

Phase 2 — Audio & Transcription (Week 3-4) 6. Implement audio device enumeration and capture 7. Build the Deepgram WebSocket client for real-time STT 8. Display live transcription in the UI with interim/final results 9. Build the transcript buffer (rolling window of last 90 seconds)

Phase 3 — Scripture Intelligence (Week 5-6) 10. Set up local SQLite Bible database (download and import a public domain translation like KJV, WEB, or BSB) 11. Build the Claude API scripture detection service 12. Build the suggestion queue UI (scripture cards with approve/dismiss) 13. Wire detection → lookup → ProPresenter push

Phase 4 — Song Lyrics (Week 7-8) 14. Build lyrics search interface 15. Implement SongSelect .usr file import 16. Build lyrics editor with section tagging (verse, chorus, bridge) 17. Push formatted lyrics to ProPresenter playlist

Phase 5 — Polish & Ship (Week 9-10) 18. Auto mode with confidence thresholds 19. Emergency clear button 20. Error handling, reconnection logic, edge cases 21. Package with electron-builder for Windows and macOS 22. User documentation

10. Development Guidelines

10.1 Architecture Principles

1. **Separate concerns across Electron's process model.** All network I/O, file system access, and API calls happen in the main process. The renderer process handles only UI. Communication happens through typed IPC channels.
2. **Service-oriented main process.** Each major feature (ProPresenter, STT, Scripture Detection, Lyrics) is an independent service class. Services communicate through events, not direct references. This makes testing and replacing components easy.
3. **Defensive coding for live services.** This software will run during live church services. Every external call (API, WebSocket, ProPresenter) must have timeouts, retry logic, and graceful fallbacks. A crash during a service is unacceptable.
4. **Operator always has control.** Never take an action without giving the operator a way to override it. Even in "auto mode," provide an instant dismiss/clear mechanism.

10.2 Electron-Specific Guidelines

```
// preload.ts — expose safe IPC to renderer
import { contextBridge, ipcRenderer } from 'electron';
```

```

contextBridge.exposeInMainWorld('api', {
  propresenter: {
    connect: (config) => ipcRenderer.invoke('pp:connect', config),
    getStatus: () => ipcRenderer.invoke('pp:status'),
    triggerSlide: (id, index) => ipcRenderer.invoke('pp:trigger', id, index),
    clearAll: () => ipcRenderer.invoke('pp:clear'),
    onStatusChange: (cb) => ipcRenderer.on('pp:status-changed', (_, data) => cb(d
  },
  audio: {
    getDevices: () => ipcRenderer.invoke('audio:devices'),
    startCapture: (deviceId) => ipcRenderer.invoke('audio:start', deviceId),
    stopCapture: () => ipcRenderer.invoke('audio:stop'),
    onLevel: (cb) => ipcRenderer.on('audio:level', (_, level) => cb(level)),
  },
  scripture: {
    onSuggestion: (cb) => ipcRenderer.on('scripture:suggestion', (_, ref) => cb(r
    approve: (id) => ipcRenderer.invoke('scripture:approve', id),
    dismiss: (id) => ipcRenderer.invoke('scripture:dismiss', id),
    search: (query) => ipcRenderer.invoke('scripture:search', query),
  },
  transcription: {
    onTranscript: (cb) => ipcRenderer.on('stt:transcript', (_, seg) => cb(seg)),
  },
});

```

10.3 Error Handling Strategy

```

// Every external service call should follow this pattern:
class ProPresenterClient {
  private retryCount = 0;
  private maxRetries = 3;

  async triggerSlide(presentationId: string, slideIndex: number): Promise<boolean>
  try {
    const response = await axios.post(
      `http://${this.host}:${this.port}/v1/presentation/${presentationId}/trigg
      { index: slideIndex },
      { timeout: 3000 }
    );
    this.retryCount = 0;
    return true;
  } catch (error) {
    this.retryCount++;
    this.emit('error', { service: 'propresenter', error, retryCount: this.retry

    if (this.retryCount < this.maxRetries) {

```

```

    await this.delay(1000 * this.retryCount); // exponential backoff
    return this.triggerSlide(presentationId, slideIndex);
  }

  this.emit('connectionLost');
  return false;
}
}
}

```

10.4 Scripture Detection Prompt Engineering

The quality of your scripture detection depends heavily on your Claude API prompt. Here are guidelines:

1. **Send context, not single sentences.** Always send 60-120 seconds of transcript context. Ministers often build up to a reference over several sentences.
2. **Use a system prompt that primes for biblical literacy.** Include a list of all 66 books with common abbreviations and alternate names.
3. **Request structured JSON output.** Always request machine-parseable output, not prose.
4. **Set a confidence threshold.** Start with 0.7 for auto-display and 0.4 for suggestion-only. Tune based on your minister’s speaking style.
5. **Debounce detection calls.** Don’t call Claude on every transcript update. Batch every 5-10 seconds, or trigger on sentence boundaries.
6. **Cache recent detections.** If John 3:16 was detected 30 seconds ago, don’t detect it again from residual transcript context.

10.5 Testing Strategy

Layer	Tool	What to Test
Unit tests	Vitest	Scripture parsing, Bible reference normalization, slide formatting
Integration tests	Vitest + mocks	ProPresenter client, Deepgram client, Claude API client
E2E tests	Playwright	Full operator workflow: connect → listen → detect → approve → project

Manual testing	Live service dry runs	Run alongside ProPresenter with recorded sermons before going live
----------------	-----------------------	--

10.6 Performance Guidelines

- **Keep the renderer lightweight.** Don't process audio or run API calls in the renderer. Use IPC.
- **Throttle UI updates.** Transcription generates many events per second. Batch UI updates at 10-15fps.
- **Use streaming for Claude API calls.** Stream the response so the operator sees results appear progressively.
- **Pool database connections.** Use a single better-sqlite3 instance, which is synchronous and fast.
- **Monitor memory.** Electron apps can leak memory. Use the `--max-old-space-size` flag if needed and periodically trim transcript history.

10.7 Security Guidelines

- **Never ship API keys in the binary.** Store them in electron-store (encrypted at rest) and let users enter their own keys.
- **ProPresenter API has no auth by default.** Ensure the app only communicates on the local network.
- **Validate all data from ProPresenter.** The API can return unexpected data; always validate before processing.
- **CCLI compliance.** Always display copyright information when projecting song lyrics. Your CCLI license requires this.

11. Bible Database Setup

For instant offline scripture lookups, bundle a local SQLite database.

Recommended Source

The **Berean Standard Bible (BSB)** is a modern, readable English translation released into the public domain (April 2023). It can be used without restriction. For KJV, the text is also public domain.

Database Schema

```
CREATE TABLE translations (  
  id TEXT PRIMARY KEY,      -- "KJV", "BSB", "WEB"  
  name TEXT NOT NULL,  
  language TEXT DEFAULT 'en',  
  is_default BOOLEAN DEFAULT 0  
);  
  
CREATE TABLE books (  
  id INTEGER PRIMARY KEY,    -- 1-66  
  name TEXT NOT NULL,        -- "Genesis"  
  abbreviation TEXT NOT NULL, -- "Gen"  
  testament TEXT NOT NULL    -- "OT" or "NT"  
);  
  
CREATE TABLE verses (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  translation_id TEXT NOT NULL,  
  book_id INTEGER NOT NULL,  
  chapter INTEGER NOT NULL,  
  verse INTEGER NOT NULL,  
  text TEXT NOT NULL,  
  FOREIGN KEY (translation_id) REFERENCES translations(id),  
  FOREIGN KEY (book_id) REFERENCES books(id)  
);  
  
CREATE INDEX idx_verses_lookup ON verses(translation_id, book_id, chapter, verse)  
CREATE INDEX idx_verses_fulltext ON verses(text); -- For quote matching
```

Data Sources

- **Free Use Bible API** (helloao/bible-api on GitHub) — MIT-licensed, 1000+ translations, JSON format
- **scrollmapper/bible_databases** on GitHub — MySQL, SQLite, XML, CSV, JSON formats
- **bible-api.com** — Free REST API, no auth, public domain translations (WEB, KJV, ASV)
- **API.Bible** (scripture.api.bible) — Free for non-commercial, 2500+ versions (requires API key)

12. Song Lyrics Integration Details

The CCLI/SongSelect Situation

CCLI has retired their SongSelect API Partner Program and is no longer accepting new API partners. This means direct API integration is not possible for new developers. However, you have alternatives:

- 1. **Import SongSelect .usr files** — Users can download songs from songselect.ccli.com as .usr files. Your app can parse these files and import them into ProPresenter.
- 2. **Import from ProPresenter’s existing library** — If the church already has songs in ProPresenter, your app can read them via the ProPresenter API.
- 3. **Manual entry with smart formatting** — Build a lyrics editor that accepts pasted lyrics and auto-detects sections (verse, chorus, bridge) based on common patterns.
- 4. **Open source hymn databases** — Public domain hymns (pre-1928) are freely available in structured formats.

.usr File Parser (SongSelect Export Format)

SongSelect exports lyrics as `.usr` files, which are plain text with a specific format that includes metadata fields and lyrics sections. Your importer would parse these fields, extract the title, author, copyright info, CCLI number, and the lyrics sections, then convert them to ProPresenter-compatible presentations.

13. Risks & Mitigations

Risk	Impact	Mitigation
ProPresenter API changes or breaks	Can’t push content	Abstract the PP client behind an interface; version-lock your integration.
Deepgram service outage	No transcription	Implement fallback to local Whisper (slower) or manual-only mode.
Claude API latency spike	Slow scripture detection	Cache common references locally; use Haiku model for faster responses when Sonnet is slow.
False positive scripture detection	Wrong verse projected during service	Always default to suggestion mode (operator approval). Auto mode only after tuning.
Audio quality issues	Poor transcription accuracy	Use direct soundboard feed, not a room mic. Add audio level monitoring.

Minister speaks in non-English	STT fails or inaccurate	Configure Deepgram for the correct language; Claude supports multilingual detection.
Network disconnection to ProPresenter	Can't control projections	Auto-reconnect with exponential backoff; visual and audio alerts to operator.
CCLI/copyright violations	Legal risk for the church	Always display copyright info with lyrics; respect license terms.

14. Estimated API Costs (Per Sunday Service)

Assuming a 45-minute sermon with continuous audio:

Service	Usage	Estimated Cost
Deepgram STT	45 min × \$0.0077/min	~\$0.35
Claude API (Sonnet)	~50 detection calls × ~1K tokens each	~\$0.50
Bible API	Free tier	\$0.00
Total per service		~\$0.85
Monthly (4 services)		~\$3.40

These costs are minimal for a church budget. Using Claude Haiku instead of Sonnet would reduce the Claude cost by roughly 90%.

15. Resources & References

ProPresenter API

- Official API docs: <https://openapi.propresenter.com/>
- Community API documentation: <https://github.com/jeffmikels/ProPresenter-API>
- Node.js ProPresenter library: <https://github.com/utopiantools/node-propresenter>
- ProPresenter MCP server (reference implementation): <https://github.com/Marc416/propresenter-mcp>

- TCP/IP API guide: <https://support.renewedvision.com/hc/en-us/articles/31606866768147>

Bible APIs & Data

- API.Bible: <https://scripture.api.bible/>
- Free Use Bible API: <https://github.com/AoLabDev/free-use-bible-api>
- bible-api.com: <https://bible-api.com/>
- Bible databases (SQLite/JSON): https://github.com/scrollmapper/bible_databases
- ESV API: <https://api.esv.org/> (non-commercial only)

Speech-to-Text

- Deepgram: <https://deepgram.com/> (recommended for real-time)
- Deepgram Node SDK: <https://github.com/deepgram/deepgram-node-sdk>
- OpenAI Whisper (self-hosted fallback): <https://github.com/openai/whisper>

Electron Development

- electron-vite: <https://electron-vite.org/>
- electron-forge: <https://www.electronforge.io/>
- Electron docs: <https://www.electronjs.org/docs>

Claude API

- Anthropic SDK: <https://github.com/anthropics/anthropic-sdk-node>
- API docs: <https://docs.anthropic.com/>

This document was prepared as a comprehensive project specification. Start with Phase 1 (Foundation) and iterate. Ship a working MVP before adding advanced features. Test extensively with recorded sermons before using in a live service.